

You are hitting on a central, fascinating headache in proof engineering: **the total lack of monotonicity between syntactic edits and logical structure**.

In standard software engineering, a minor syntactic change (like swapping a loop direction or modifying an assignment) might map cleanly to a small diff in a syntax tree. But in formal verification, changing a single line of concrete code or an interactive proof tactic script can completely break the downstream proof state, causing an exponential explosion or a complete structural divergence in the logical proof obligations.

While Kiran Gopinathan's landmark paper on **Sisyphus** (*"Mostly Automated Proof Repair for Verified Libraries"*) addresses the broader problem of invariant drift and proof repair, her methodology tackles exactly what you are experiencing—the fact that you **cannot** simply rely on concrete tactic diffs or basic syntax alignment to repair a proof (Gopinathan et al., 2023).

Instead of treating proof repair as a syntactic text-diff problem, her approach pivots entirely to **semantic alignment, runtime behavior, and logical feedback** (Gopinathan et al., 2023). Here is how her work circumvents the lack of correlation between concrete tactic syntax and logical structure:

1. Relational Tracing Over Syntactic Diffs

Because a small change in concrete syntax yields unpredictable changes in the logical obligations, Sisyphus does not try to align the old proof script text to the new program text directly. Instead, it uses **dynamic program alignment** (Gopinathan et al., 2023).

- It runs instrumented, concrete executions of the old program (p) and the new mutated program (p').
- By recording the values flowing through variables during execution, it discovers how the intermediate states of the two programs actually map to one another *semantically*, regardless of how radically different the code syntax or the corresponding proof scripts look.

2. Bypassing the Tactic Script via Invariant Synthesis

When code evolves, the structural layout of the verification predicates (like Separation Logic heap predicates) shifts unpredictably. Sisyphus recognizes that fixing the *text* of a tactic script is a losing battle. Instead, it shifts the problem to **loop invariant repair**.

- It takes the semantic mapping generated from the dynamic alignment and uses it to translate the *old* proven invariants into a candidate pool for the *new* program.
- It abstracts away the concrete step-by-step tactics, focusing purely on identifying the correct underlying logical properties that must hold true at specific execution checkpoints.

3. Pruning the Space with "Proof-Driven Testing"

This is where your concern about non-monotonicity is directly solved. If you guess a new invariant or a new step in a proof, how do you know if you are getting "closer" to a valid proof? In standard syntax, you can't—there's no continuous gradient.

To bridge this gap, her work introduces **Proof-Driven Testing** (Gopinathan et al., 2023).

- Coq proof objects are constructive (via the Curry-Howard correspondence). They aren't just text; they are executable terms.
- Sisyphus takes the original proof certificates, extracts the executable constructive content, and uses them as a dynamic testing suite against candidate invariants.

- Instead of checking if a *tactic script* passes or fails, it tests whether the *logical data structure of the old proof certificate* can still validly consume or match the runtime counterexamples of the new program version. This allows the tool to drastically prune wild, structurally broken paths without having to invoke the slow, unpredictable automated theorem prover (ATP) loop for every syntactic guess.

Why Your Intuition About Non-Monotonicity is Spot-on

Your realization that you can't assume a correlated relationship between code/tactic diffs and logical structure is the exact reason why syntax-based or purely LLM-based proof repair frequently hits a brick wall. A language model might look at a tactic diff and think, "*Oh, I'll just change induction n to induction m ,*" only to find that the entire induction hypothesis requires a completely restructured heap invariant.

Kiran's work with Sisyphus argues that the only way forward is to decouple the repair mechanism from the concrete tactic script entirely (Gopinathan et al., 2023). By focusing on the **underlying semantics of the execution** and using **constructive proof objects as executable tests**, you bypass the erratic, non-monotonic world of tactic syntax and operate instead on the underlying logical properties (Gopinathan et al., 2023).

References

Gopinathan, K., Keoliya, M., & Sergey, I. (2023). Mostly automated proof repair for verified libraries. *Proceedings of the ACM on Programming Languages*, 7(PLDI), 25–49.
<https://doi.org/10.1145/3591221> Cited by: 22